DOCKER

A dark-mode beginner's field guide • concepts, commands, and practical examples

What you'll learn: Docker basics, images and containers, essential commands, Dockerfiles, ports, volumes, environment variables, networks, Compose, debugging, and ML/LLM workflows. Code blocks use high-contrast text for readability.

1. What is Docker?

Docker packages an application with the runtime, libraries, and configuration it needs. This makes it easier to run the app consistently on different machines. An **image** is the reusable package; a **container** is a running instance of that image.

Term	Plain-English meaning	Example
Image	Reusable template/package.	python:3.12-slim
Container	An isolated process created from an image.	A running Python app
Dockerfile	Instructions for building an image.	Install packages, copy code
Registry	Place to store and download images.	Docker Hub
Volume	Storage that can outlive a container.	Persist results

Why use Docker?

It reduces “works on my machine” problems caused by different runtime versions and dependencies. Containers share the host OS kernel, so they are generally lighter than full virtual machines.

Install and verify

```
docker --version
docker info
docker run hello-world
```

These commands show the CLI version, check whether Docker Engine is reachable, and run a test container. Docker Desktop is a common setup on Windows and macOS.

2. Images and containers

An image is a template. You can create multiple containers from one image; each container has its own runtime state.

Pull, list, run, inspect

```
docker pull python:3.12-slim
docker images

docker run --name demo-python python:3.12-slim python --version

docker ps
docker ps -a
docker inspect demo-python
```

docker ps lists running containers. **docker ps -a** also lists stopped ones. The Python container prints its version and exits.

Run an interactive container

```
docker run --rm -it python:3.12-slim python
>>> print("Hello from Docker")
>>> exit()
```

-it gives you an interactive terminal. **--rm** removes the container when it exits.

Container lifecycle

```
docker start demo-python
docker stop demo-python
docker restart demo-python
docker rm demo-python
docker rmi python:3.12-slim
```

Stop a container before removing it. Docker may refuse to remove an image if existing containers still depend on it.

Common flags

Flag	Meaning
-d	Run in background (detached mode).
-p 8080:80	Map host port 8080 to container port 80.
-v	Mount a volume or host folder.
-e KEY=value	Set an environment variable.
--name app	Give the container a readable name.
--rm	Remove the container when it exits.

Important: files written only inside a container may be lost when the container is removed. Use a volume or bind mount for data you need to keep.

3. Build your first image with a Dockerfile

A Dockerfile is a text file containing the instructions Docker uses to build an image.

Step 1 — create the app

Create a folder named **docker-demo**. Inside it, create **app.py**:

```
print("Hello! This app runs inside a container.")
```

Create another file named **Dockerfile** (no extension):

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

Step 2 — build and run

```
# Run from inside the docker-demo folder
docker build -t hello-docker:1.0 .
docker run --name hello-run hello-docker:1.0
docker ps -a
docker rm hello-run
```

The dot means “use the current folder as the build context.” **-t** gives the image a name and tag.

Dockerfile instructions

Instruction	Purpose
FROM	Choose a base image.
WORKDIR	Set the working directory inside the image.
COPY	Copy files from the build context into the image.
RUN	Execute a command during image building, such as installing packages.
CMD	Default command when a container starts.
EXPOSE	Documents an intended port; does not publish it.
ENV	Set environment variables.

Install dependencies (example)

```
# requirements.txt contains:
requests==2.32.3

# Add to Dockerfile:
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Build-cache tip: copy dependency files and install dependencies before copying frequently changing source code. Docker may reuse cached layers and speed up rebuilds.

4. Ports, environment variables, storage, and networks

Ports: reach your app from your computer

If an app listens on port 8000 inside a container, publish it with a host-to-container mapping.

```
docker run -d --name web -p 8000:8000 my-web-image
docker logs -f web
docker stop web
docker rm web
```

The format is **HOST_PORT:CONTAINER_PORT**. The app should usually listen on `0.0.0.0` inside the container, not only `127.0.0.1`. Replace `my-web-image` with your own image.

Environment variables

```
docker run --rm -e APP_MODE=dev my-image
```

The app can read `APP_MODE` from its environment. Never bake passwords, API keys, or tokens into a Dockerfile or image.

Persistent storage

Type	Example	Best for
Named volume	-v results:/app/results	Docker-managed persistent data.
Bind mount	-v "${PWD}:/app"	Share a specific host folder with a container.

```
docker volume create results
docker run --rm -v results:/app/results my-image

# Example bind mount (Bash-style path)
docker run --rm -v "${PWD}:/app" -w /app python:3.12-slim ls
```

Host path quoting varies across Bash, PowerShell, and Command Prompt.

Networking

```
docker network create app-net
docker run -d --name api --network app-net my-api-image
docker network ls
```

Containers on the same user-defined network can often reach each other by container name. For example, a client may use `http://api:8000` if the API listens on port 8000.

5. Docker Compose and troubleshooting

Docker Compose describes related services in a YAML file and runs them together.

Example compose.yaml

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      APP_MODE: development
    volumes:
      - ./data:/app/data
```

Save as **compose.yaml** beside the Dockerfile. This example assumes your app listens on port 8000 and uses `/app/data` for data.

Compose commands

```
docker compose up --build
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down
```

up --build builds and starts services; **-d** runs them in the background; **logs -f** streams logs; **down** stops and removes the project's containers and network. **docker compose down -v** also removes named volumes, so use it carefully.

Debugging commands

```
docker ps -a
docker logs --tail 100 my-container
docker exec -it my-container sh
docker stats
docker system df
```

Problem	First thing to check
Container exits	docker ps -a, then docker logs CONTAINER
App unreachable	Check port mapping, app port, and bind address.
Build error	Read the first useful error; verify Dockerfile and COPY paths.
Changes not visible	Rebuild the image or use a bind mount.
Disk usage	docker system df

`docker exec` works only on a running container, and some minimal images do not contain a shell.

6. ML/LLM workflows, good habits, and cheat sheet

A common data/evaluation pattern

```
docker run --rm \
  -v "${PWD}/dataset:/data:ro" \
  -v "${PWD}/results:/results" \
  -e SEED=42 \
  my-eval-image:1.0
```

The app should read inputs from `/data` and write outputs to `/results`. `:ro` makes the dataset mount read-only. Create the host results folder first if needed. GPU use requires compatible hardware, drivers, and setup; Docker alone does not enable it.

Good habits

Habit	Why it helps
Use explicit image tags	Avoid unexpected changes from floating defaults.
Add .dockerignore	Keep .git, caches, datasets, and secrets out of the build context.
Keep images small	Builds and transfers can be faster.
Avoid root where practical	Reduce the impact of a compromised app.
Never bake secrets into images	Image layers may retain copied secrets.
Clean selectively	Avoid deleting volumes or images you still need.

Quick command cheat sheet

Task	Command
Download image	docker pull IMAGE
Build image	docker build -t NAME:TAG .
Run in background	docker run -d --name NAME IMAGE
List containers	docker ps | docker ps -a
View logs	docker logs -f NAME
Open a shell	docker exec -it NAME sh
Stop and remove	docker stop NAME ; docker rm NAME
List images / volumes	docker images | docker volume ls
Compose start / stop	docker compose up --build | docker compose down

Practice: Build `hello-docker`, change the message, rebuild it, then mount a host folder and save an output file. If something fails, start with `docker ps -a` and `docker logs`.

Mental model: Dockerfile → build → image → run → container. Add ports for access, volumes for persistence, environment variables for configuration, and Compose for multiple services.